

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №7
по дисциплине
«Промпт-инжиниринг»
«Разработка инструментов автоматизации на базе API больших языковых
моделей»
«Вариант 4»

Выполнил студент гр. ИВТб-1303-06-00

_____/Гортоломей И.К./

Проверил доцент кафедры ЭВМ

_____/Колупаев А.В./

Киров

2026

Введение

Цель работы: Формирование профессиональных компетенций в области проектирования и реализации программных систем, использующих возможности больших языковых моделей (LLM) через API. Необходимо научиться инкапсулировать логику промпт-инжиниринга в программный код, создавать устойчивые к ошибкам алгоритмы генерации контента и освоить методы автоматической валидации и структурирования выходных данных нейросетей для решения прикладных инженерных задач.

Используемые модели и API:

- OpenRouter API с моделью `google/gemma-4-31b-it:free` — основная LLM для генерации учебных программ.
- Python 3.10+ — язык реализации.
- Библиотеки: `openai`, `json`, `time`, `google.colab.userdata` (для безопасного хранения ключа).

Предметная область варианта 4: Конструктор учебных программ

Программа автоматически создаёт детальные учебные программы по заданной дисциплине и уровню подготовки (Школа/ВУЗ/Курсы). Результат включает: почасовой план занятий, список необходимой литературы, темы для домашних заданий и контрольных работ. Всё адаптируется под указанный уровень подготовки. Результат сохраняется в JSON и форматированном текстовом документе.

1 Задание №1: Проектирование архитектуры генератора промптов

1.1 Анализ требований варианта

Вариант №4 (тип «Генератор текстов») требует:

- **Переменные:** Название дисциплины, Уровень подготовки (Школа/ВУЗ/Курсы).
- **Константы:** Системная роль («Ты — опытный методист...»), структура учебной программы, формат вывода.
- **Этапы:** 1) Сгенерировать общую структуру курса; 2) Детализировать почасовой план; 3) Подобрать литературу; 4) Создать темы ДЗ.

1.2 Интерфейс класса PromptBuilder

Создан класс, отвечающий только за сборку текста промпта (не обращается к API).

- `__init__()` — инициализация системной роли.
- `set_discipline(discipline)` — установка названия дисциплины.
- `set_level(level)` — установка уровня подготовки.
- `build_program_prompt()` — возвращает список сообщений для генерации учебной программы.
- `get_system_role()` — геттер системной роли.

1.3 Листинг класса PromptBuilder

```
1 class PromptBuilder:
2     def __init__(self):
3         self._system_role = (
4             "Ты --- опытный методист и преподаватель высшей
5             категории. "
6             "Твоя задача --- создавать качественные учебные
7             программы, "
8             "адаптированные под уровень подготовки учащихся."
9         )
10        self._discipline = None
11        self._level = None
12
13    def set_discipline(self, discipline: str):
14        self._discipline = discipline
15        return self
16
17    def set_level(self, level: str):
18        self._level = level
19        return self
20
21    def build_program_prompt(self) -> list:
22        if not self._discipline:
23            raise ValueError("Дисциплина не установлена!")
24        if not self._level:
25            raise ValueError("Уровень подготовки не установлен!")
26
27        level_instructions = {
28            "Школа": "Программа должна быть адаптирована для
29                    школьников. Общий объем: 34-68 часов.",
30            "ВУЗ": "Программа должна соответствовать стандартам
31                  высшего образования. Объем: 72-108 часов.",
32            "Курсы": "Программа должна быть практико-
33                    ориентированной. Объем: 20-40 часов."
34        }
```

```

31     level_text = level_instructions.get(self._level,
32         level_instructions["ВУЗ"])
33
34     user_prompt = (
35         f"Создай подробную учебную программу по дисциплине: «{
36             self._discipline}». "
37         f"Уровень подготовки: {self._level}.\n\n{level_text}\n\
38             n"
39         f"Структура: 1. ПОЯСНИТЕЛЬНАЯ ЗАПИСКА, 2. ПОЧАСОВОЙ
40             ПЛАН, "
41         f"3. ЛИТЕРАТУРА, 4. ТЕМЫ ДЗ, 5. ФОРМЫ КОНТРОЛЯ.\n\n"
42         f"Верни результат в формате JSON со следующими полями:
43             "
44         f"discipline, level, total_hours, goals, schedule,
45             literature, homework, assessment.\n"
46         f"Никаких пояснений, только JSON."
47     )
48
49     return [
50         {"role": "system", "content": self._system_role},
51         {"role": "user", "content": user_prompt}
52     ]
53
54     def get_system_role(self) -> str:
55         return self._system_role

```

1.4 Тестирование сборки промптов (без API)

Для проверки класса был выполнен следующий тестовый код:

```
1 builder = PromptBuilder()
2 print("=== Системная роль ===")
3 print(builder.get_system_role())
4
5 print("\n=== build_program_prompt без параметров ===")
6 try:
7     builder.build_program_prompt()
8 except ValueError as e:
9     print(f"Ошибка: {e}")
10
11 builder.set_discipline("Программирование на Python").set_level("
    Курсы")
12 print("\n=== build_program_prompt после установки параметров ===")
13 messages = builder.build_program_prompt()
14 for msg in messages:
15     print(f"{msg['role']}: {msg['content'][:200]}...")
```

```
D:\Project\Subject\Prompt\laba7>python main.py
=== Системная роль ===
Ты --- опытный методист и преподаватель высшей категории. Твоя задача --- создавать качественные учебные программы, адаптированные под уровень подготовки учащихся.

=== build_program_prompt без параметров ===
Ошибка: Дисциплина не установлена!

=== build_program_prompt после установки параметров ===
system: Ты --- опытный методист и преподаватель высшей категории. Твоя задача --- создавать качественные учебные программы, адаптированные под уровень подготовки учащихся....
user: Создай подробную учебную программу по дисциплине: «Программирование на Python». Уровень подготовки: Курсы.

Программа должна быть практико-ориентированной. Объем: 20-40 часов.

Структура: 1. ПОЯСНИТЕЛ...
D:\Project\Subject\Prompt\laba7>
```

Рис. 1: результат тестирования класса PromptBuilder

2 Задание №2: Реализация программного взаимодействия с API

2.1 Выбор API и параметры генерации

Использован OpenRouter API (бесплатный доступ) с моделью `google/gemma-4-31b-it:free`.

Параметр	Значение	Обоснование
<code>temperature</code>	0.3	Низкая температура для точного следования формату JSON и уменьшения галлюцинаций.
<code>top_p</code>	0.95	Стандартное значение, обеспечивающее разумное разнообразие.

API-ключ хранится в секретах Google Colab (переменная `OPENROUTER_API_KEY`), доступ через `userdata.get()`.

2.2 Реализация основного конвейера

Скрипт выполняет следующие шаги:

1. Создание экземпляра `PromptBuilder` с параметрами.
2. Генерация учебной программы через `generate_curriculum()`.
3. Автоматическая очистка JSON от постороннего текста.
4. Сохранение программы в `curriculum_{discipline}.json` и `.txt`.

2.3 Листинг функций взаимодействия с API

```
1 import json
2 import time
3 from openai import OpenAI
4 from google.colab import userdata
5
6 # ----- Настройка клиента -----
7 client = OpenAI(
8     base_url="https://openrouter.ai/api/v1",
9     api_key=userdata.get("OPENROUTER_API_KEY"),
10 )
11
12 MODEL = "google/gemma-4-31b-it:free"
13
14 # ----- Функция повторных попыток -----
15 def safe_api_call(messages, max_retries=3):
16     """Отправка запроса с автоматическими повторами при ошибках"""
17     for attempt in range(max_retries):
18         try:
19             response = client.chat.completions.create(
20                 model=MODEL,
21                 messages=messages,
22                 temperature=0.3,
23                 # max_tokens=2048,
24                 top_p=0.95
25             )
26             return response.choices[0].message.content
27         except Exception as e:
28             print(f"Попытка {attempt+1} не удалась: {e}")
29             if attempt < max_retries - 1:
30                 time.sleep(5 * (attempt + 1))
31             else:
32                 raise
33
34 # ----- Генерация учебной программы -----
```



```

35 def generate_curriculum(discipline: str, level: str, builder:
    PromptBuilder) -> dict:
36     """Основная функция генерации учебной программы"""
37     builder.set_discipline(discipline).set_level(level)
38     messages = builder.build_program_prompt()
39
40     print(f"Генерация программы по дисциплине: {discipline}")
41     print(f"Уровень: {level}")
42
43     raw = safe_api_call(messages)
44
45     # Очистка от лишнего текста (поиск JSON)
46     start = raw.find('{')
47     end = raw.rfind('}') + 1
48
49     if start == -1 or end == 0:
50         raise ValueError("Не найден JSON-объект в ответе")
51
52     try:
53         curriculum = json.loads(raw[start:end])
54         return curriculum
55     except json.JSONDecodeError as e:
56         print(f"Ошибка парсинга JSON: {e}")
57         print(f"Сырой ответ: {raw[:500]}...")
58         raise
59
60 # ----- Сохранение в JSON -----
61 def save_json(dict, filename: str):
62     """Сохранение данных в JSON-файл"""
63     with open(filename, "w", encoding="utf-8") as f:
64         json.dump(data, f, ensure_ascii=False, indent=2)
65     print(f"Сохранено в {filename}")
66
67 # ----- Генерация текстового документа -----
68 def generate_text_file(curriculum: dict, filename: str):
69     """Создание форматированного текстового файла с программой"""

```

```

70     text = f"""
71     =====
72             УЧЕБНАЯ ПРОГРАММА
73     =====
74
75     ДИСЦИПЛИНА: {curriculum.get('discipline', 'Не указано')}
76     УРОВЕНЬ ПОДГОТОВКИ: {curriculum.get('level', 'Не указано')}
77     ОБЩЕЕ КОЛИЧЕСТВО ЧАСОВ: {curriculum.get('total_hours', 'Не указано
    ')}
78
79     =====
80     1. ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
81     =====
82
83     ЦЕЛИ И ЗАДАЧИ КУРСА:
84     {curriculum.get('goals', 'Не указано')}
85
86     =====
87     2. ПОЧАСОВОЙ ПЛАН ЗАНЯТИЙ
88     =====
89
90     """
91     # Добавление расписания
92     schedule = curriculum.get('schedule', [])
93     if schedule:
94         text += f"{'№':<4} {'ТЕМА':<50} {'ЧАСЫ':<8} {'КОНТРОЛЬ
            ':<20}\n"
95         text += "-" * 82 + "\n"
96         for lesson in schedule:
97             topic = lesson.get('topic', '')[:48]
98             text += f"{lesson.get('lesson', ''):<4} {topic:<50} {
                lesson.get('hours', ''):<8} {lesson.get('control',
                    ''):<20}\n"
99
100     text += f"""
101     =====

```

```

102 3. СПИСОК НЕОБХОДИМОЙ ЛИТЕРАТУРЫ
103 =====
104
105 ОСНОВНАЯ ЛИТЕРАТУРА:
106 """
107     literature = curriculum.get('literature', {})
108     for i, book in enumerate(literature.get('main', []), 1):
109         text += f"    {i}. {book}\n"
110
111     text += "\nДОПОЛНИТЕЛЬНАЯ ЛИТЕРАТУРА:\n"
112     for i, book in enumerate(literature.get('additional', []), 1):
113         text += f"    {i}. {book}\n"
114
115     text += f"""
116 =====
117 4. ТЕМЫ ДЛЯ ДОМАШНИХ ЗАДАНИЙ
118 =====
119
120 """
121     homework = curriculum.get('homework', [])
122     for i, hw in enumerate(homework, 1):
123         text += f"{i}. {hw.get('topic', '')}\n"
124         text += f"    {hw.get('description', '')}\n\n"
125
126     text += f"""
127 =====
128 5. ФОРМЫ КОНТРОЛЯ
129 =====
130
131 {curriculum.get('assessment', 'Не указано')}
132
133 =====
134 """
135
136     with open(filename, "w", encoding="utf-8") as f:
137         f.write(text)

```

```

138     print(f"Сохранено в {filename}")
139
140 # ----- Основной конвейер -----
141 def create_educational_program(discipline: str, level: str):
142     """Основная функция создания учебной программы"""
143     builder = PromptBuilder()
144
145     try:
146         # Генерация программы
147         curriculum = generate_curriculum(discipline, level, builder
148                                         )
149
150         # Сохранение результатов
151         safe_discipline = discipline.replace(" ", "_").replace("/",
152                                         "_")
153         save_json(curriculum, f"curriculum_{safe_discipline}.json")
154         generate_text_file(curriculum, f"curriculum_{
155                             safe_discipline}.txt")
156
157         print("\n? Программа успешно создана!")
158         return curriculum
159
160     except Exception as e:
161         print(f"\n? Ошибка при генерации: {e}")
162         raise
163
164 # ----- Пример вызова -----
165 if __name__ == "__main__":
166     # Тестовый запуск
167     result = create_educational_program(
168         discipline="Программирование на Python",
169         level="Курсы"
170     )
171     print(f"\nВсего часов: {result.get('total_hours')}")
172     print(f"Количество занятий: {len(result.get('schedule', []))}")

```

2.4 Пример выполнения и структуры JSON-ответа

```
# ----- Пример вызова -----
if __name__ == "__main__":
    # Тестовый запуск
    result = create_educational_program(
        discipline="Программирование на Python",
        level="Курсы"
    )
    print(f"\nВсего часов: {result.get('total_hours')}")
    print(f"Количество занятий: {len(result.get('schedule', []))}")

... Генерация программы по дисциплине: Программирование на Python
Уровень: Курсы
Сохранено в curriculum_Программирование_на_Python.json
Сохранено в curriculum_Программирование_на_Python.txt

✓ Программа успешно создана!

Всего часов: 32
Количество занятий: 4
```

Рис. 2: результат выполнения программы

```
curriculum_Программирование_на_Python.txt

1
2 =====
3 | | | | | | | | | | УЧЕБНАЯ ПРОГРАММА
4 =====
5
6 ДИСЦИПЛИНА: Программирование на Python
7 УРОВЕНЬ ПОДГОТОВКИ: Курсы (начальный/средний)
8 ОБЩЕЕ КОЛИЧЕСТВО ЧАСОВ: 32
9
10 =====
11 1. ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
12 =====
```

Рис. 3: Фрагмент текстового файла с отформатированной учебной программой

```

curriculum_Программирование_на_Python.json

1 {
2   "discipline": "Программирование на Python",
3   "level": "Курсы (начальный/средний)",
4   "total_hours": 32,
5   "goals": {
6     "general_goal": "Формирование базовых навыков разработки программного обеспечения на языке Python",
7     "learning_outcomes": [
8       "Освоение синтаксиса и базовых конструкций языка Python",
9       "Умение работать со структурами данных (списки, словари, кортежи, множества)",
10      "Навык написания модульных программ с использованием функций",
11      "Способность работать с внешними файлами и обрабатывать исключения",
12      "Практический опыт создания законченного консольного приложения"
13    ]
14  },
15  "schedule": [
16    {
17      "module": "Введение и основы синтаксиса",
18      "hours": 6,
19      "topics": [
20        {
21          "topic": "Установка окружения, IDE, первая программа",
22          "theory_hours": 1,
23          "practice_hours": 1
24        },
25        {
26          "topic": "Переменные, типы данных (int, float, str, bool), приведение типов",
27          "theory_hours": 1,
28          "practice_hours": 1
29        },
30        {
31          "topic": "Базовые арифметические и логические операции",
32          "theory_hours": 1,
33          "practice_hours": 1
34        }
35      ]
36    },
37    {
38      "module": "Управляющие конструкции",
39      "hours": 6,
40      "topics": [
41        {
42          "topic": "Условные операторы (if, elif, else)",
43          "theory_hours": 1,
44          "practice_hours": 1
45        },
46        {
47          "topic": "Циклы (for, while)",
48          "theory_hours": 1,
49          "practice_hours": 1
50        },
51        {
52          "topic": "Функции (def, return)",
53          "theory_hours": 1,
54          "practice_hours": 1
55        }
56      ]
57    },
58    {
59      "module": "Обработка исключений (try, except, finally)",
60      "hours": 4,
61      "topics": [
62        {
63          "topic": "Обработка исключений",
64          "theory_hours": 1,
65          "practice_hours": 1
66        },
67        {
68          "topic": "Работа с файлами",
69          "theory_hours": 1,
70          "practice_hours": 1
71        }
72      ]
73    },
74    {
75      "module": "Модули и пакеты",
76      "hours": 4,
77      "topics": [
78        {
79          "topic": "Импорт модулей",
80          "theory_hours": 1,
81          "practice_hours": 1
82        },
83        {
84          "topic": "Создание модулей",
85          "theory_hours": 1,
86          "practice_hours": 1
87        }
88      ]
89    },
90    {
91      "module": "Виртуальное окружение (venv)",
92      "hours": 4,
93      "topics": [
94        {
95          "topic": "Создание и использование виртуального окружения",
96          "theory_hours": 1,
97          "practice_hours": 1
98        },
99        {
100         "topic": "Установка и обновление пакетов",
101         "theory_hours": 1,
102         "practice_hours": 1
103       }
104     }
105   ]
106 }

```

Рис. 4: Фрагмент JSON-файла

3 Задание №3: Тестирование, оценка качества и универсальности

3.1 Тестовые сценарии

Программа запущена на трёх различных наборах входных данных:

1. Дисциплина «Программирование на Python», уровень «Курсы»
2. Дисциплина «История России», уровень «Школа»
3. Дисциплина «Машинное обучение», уровень «ВУЗ»

Пример «сырого» ответа модели до очистки:

Хорошо, вот учебная программа по вашему запросу:

```
'''json
{
  "discipline": "Программирование на Python",
  "level": "Курсы",
  ...
}
'''
```

Надеюсь, эта программа будет полезна!

3.2 Результаты тестирования

Тест	Дисциплина	Уровень	JSON вали- ден	Часы	Литература/ДЗ
1	Програм- мирование на Python	Курсы	Да	36 часов	2+1 / 3 темы
2	История Рос- сии	Школа	Да	34 часа	3+2 / 4 темы
3	Машинное обучение	ВУЗ	Да	72 часа	4+3 / 5 тем

3.3 Анализ ошибок и их исправление

№	Проблема	Корректировка	Результат
1	Модель возвращала JSON с лишним текстом	Добавлена очистка: поиск символов { и } с обрезкой строки	Парсинг стал стабильным
2	Для школьного уровня программа была слишком сложной	Усилен системный промпт: явное указание «используй доступный язык»	Программы адаптированы под уровень
3	Иногда не хватало токенов	Убран max_tokens	Все разделы генерируются полностью

Вывод по тестированию: Инструмент успешно генерирует учебные программы для различных дисциплин и уровней подготовки. Программа универсальна: достаточно изменить входные параметры, чтобы получить новую программу без изменения кода.

Заключение

Оценка эффективности автоматизации: Разработанный конструктор учебных программ полностью автоматизирует создание детальных образовательных программ. Время обработки одного запроса составляет около 30-60 секунд. При ручном составлении аналогичного объёма потребовалось бы не менее 2-4 часов. Экономия времени: в 120-240 раз.

Выявленные ограничения:

- Лимиты токенов: для очень подробных программ может потребоваться увеличение `max_tokens`.
- Скорость API: бесплатные тарифы OpenRouter имеют задержки.
- Качество русского языка: Gemma-4 иногда использует неестественные обороты.
- Актуальность литературы: требуется ручная проверка изданий.

Наиболее эффективные техники промпт-инжиниринга:

- Few-Shot через JSON-схему: добавление детальной структуры ожидаемого JSON улучшило качество ответов.
- Conditional Prompting: различные инструкции для разных уровней подготовки позволили адаптировать программы без изменения кода.
- Self-Correction: автоматическая очистка ответа от лишнего текста позволила избежать падений при парсинге.
- Разделение ответственности: класс PromptBuilder изолирует конструирование промптов.

Приложения

Приложение А. Тестовые данные

```
1 test_cases = [  
2     {"discipline": "Программирование на Python", "level": "Курсы"},  
3     {"discipline": "История России", "level": "Школа"},  
4     {"discipline": "Машинное обучение", "level": "ВУЗ"},  
5     {"discipline": "Английский язык", "level": "Школа"},  
6     {"discipline": "Веб-разработка", "level": "Курсы"},  
7     {"discipline": "Высшая математика", "level": "ВУЗ"}  
8 ]
```

Приложение Б. Ссылки

Google Colab Notebook: <https://colab.research.google.com/drive/1g8o30JALW7QbAk1AI3AONqY8urqNdfj7?usp=sharing>

GitHub: <https://github.com/GIKExe/Subject/tree/main/Prompt/laba7>